NonlinearSolve.jl

Quick Overview of NonlinearSolve.jl

- Open-source, high-performance julia package for solving nonlinear systems of equations and rootfinding problems that began in 2020.
- Provides multiple methods for solving problems, such as SimpleNewtonRaphson for small scale problems, and LevenbergMarquardt for large scale/numerically difficult problems.
- Large community, with 48 contributors over the past year, though this number is slightly inflated due to bots and users with multiple accounts.
- Community members communicate through Zulip.

Basic Rootfinding

```
using NonlinearSolve
parabola(u, p) = u[1]^2 - p
p = 1.0
prob = NonlinearProblem(parabola, .1, p)#The second argument is the initial guess, the third argument is the values of p, which are constants
sol = solve(prob, NewtonRaphson())
println("Solution: x = $(sol.u)")
```

```
Solution: x = 1.0
```

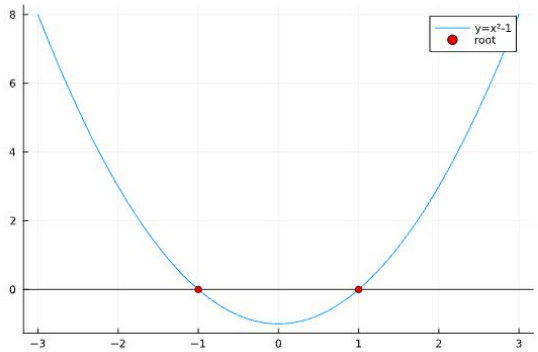
Note that `solve()` only finds one root at a time. If we wanted to find multiple roots, we could iterate through multiple initial guesses, but since we know the roots already, we can just make 2 initial guesses we know will converge to distinct roots and push the solutions to an array.

```
roots = [sol.u]  #Create an array containing one of the roots

prob2 = NonlinearProblem(parabola, -.1, p)
sol2 = solve(prob2, NewtonRaphson())
push!(roots, sol2.u)
```

```
2-element Vector{Float64}:
  1.0
 -1.0
```

```
xs = -3:.01:3
plot(xs, [parabola(x, p) for x in xs], label = "y=x²-1", legend = :topright)
hline!([0], color = :black, label = "")
scatter!(roots, [0], color = :red, label = "root")
```



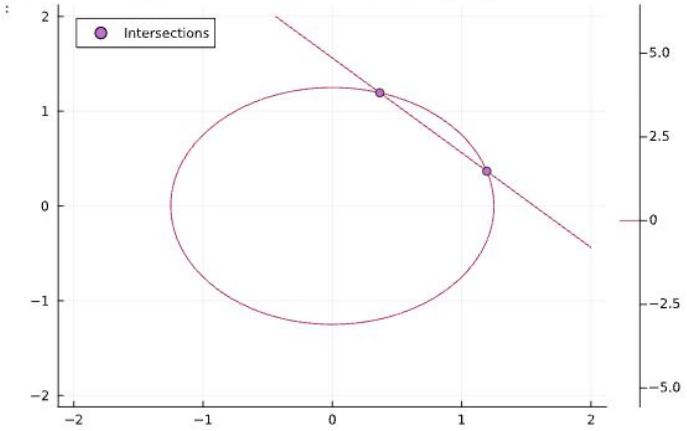
Solving A Nonlinear System of 3 Equations:

$$\begin{cases} x^2 + y^2 - z & = 0 \\ x^2 + y^2 + z^2 - 4 & = 0 \\ x + y - z & = 0 \end{cases}$$

```
x1 = 0.3670559577840663
x2 = 1.1944968550247639

y1 = 1.1944968550247639
y2 = 0.3670559577840664

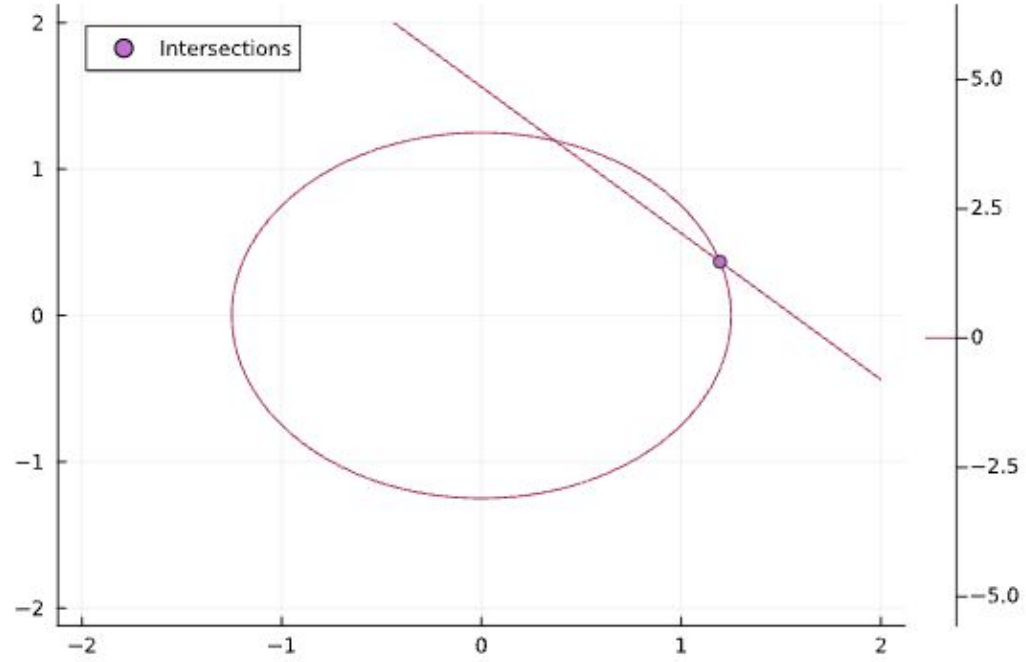
Point 1: (0.3670559577840663, 1.1944968550247639, 1.5615528128088303)
Point 2: (1.1944968550247639, 0.3670559577840664, 1.5615528128088303)
```



This image plots my manual calculation of the solution to the system.

For visualization, I plot the intersection on the plane z ≈1.562.

```
[1.194496855024764, 0.3670559577840662, 1.5615528128088303]
```




```
The absolute error is: 3.1401849173675503e-16
```

Using the methods provided to us by the library, we can obtain an accurate approximation of the intersection via Newton's Method.

Note: Each call finds one solution/root.

Questions

- I'd like to explore how the Newton's Method in this library differs from our classroom implementation.
- I only experimented with the library's NewtonRaphson method, but I'd like to look at the other methods the library provides, and determine in which scenario does each method excel.

Possible Experiment/Group Project

- Compare the number of iterations and time per iteration between the classroom implementation of Newton's Method and the method provided to us by NonlinearSolve.jl.
- Determine if Newton's method provided in the library is less likely to diverge or not. If so, what modifications does it make compared to our class implementation.